Baggage AI – A Generative AI-Powered Verified Travel Assistant

Tilak Luniya Kapil Marathe Rohita Oswal Mahima Patle Atharva Sonar

Department of Artificial Intelligence & Data Science

VP's Kamalnayan Bajaj Institute of Engineering and Technology

Introduction

Planning a trip is all about balancing different elements like where to go, your budget, how much time you have, the weather, and what you enjoy. While sites like Google Travel and TripAdvisor are great for finding popular spots, they often miss out on unique and genuine experiences. Recent innovations in Generative AI and multimodal systems have started to provide better travel advice, but they can sometimes lead to inaccuracies or outdated information due to unreliable sources like OpenStreetMapBaggage AI is here to tackle these challenges by bringing together multiple trustworthy information sources, complete with layered validation and human supervision. Designed as a modular multi-agent system using LangGraph, it combines various data retrieval methods (from OSM, travel blogs, and APIs) with real-time verification (Google Places, OpenWeather) and personalized recommendations based on memory.

This project is motivated by two main goals:

Trustworthiness – Making sure travel suggestions are verified, up-to-date, and accurate.

Diversity & Discovery – Showcasing lesser-known, culturally rich, and local attractions.

By blending AI-driven intelligence with thorough validation and personalization, Baggage AI offers reliable, user-friendly, and diverse travel suggestions that foster trust and enhance your journey.

Abstract

To guarantee precise, transparent, and verifiable travel recommendations, the system combines Self-Retrieval Augmented Generation (Self-RAG) with a multi-agent archi tecture constructed with LangGraph. It is composed of three primary modules: a Retrieval Agent that collects contextual travel information from OpenStreetMap and Google Places, a Corrective Validator that performs cross-checking using external APIs, and a Human-in-the-Loop Validator that manages outputs with low confidence or un certainty. For user convenience, both the Command-Line and Streamlit interfaces are used to display the final, verified itineraries. When compared to traditional AI travel as sistants, experimental evaluation reveals significant gains in response reliability, factual consistency, and user trust.

Keywords: Generative AI, Self-RAG, LangGraph, Travel Assistant, Human-in-the Loop, Data Verification.

Proposed System

Problem Definition:

Currently, travel assistants don't have the diversity, real-time validation, and personal ization we need, which can lead to unreliable and less authentic itineraries, ultimately affecting user experience.Although AI-powered tools have come a long way, many of them still depend on static data sources and heuristic rules, making them vulnerable to outdated or even made-up recommendations.This presents one of the biggest challenges in the TravelTech industry— trustworthiness.

The problem can therefore be stated as: "To design and implement a GenerativeAI-powered travel assistant that integrates retrieval, validation, and personalization to deliver verified, diverse, and trustworthy travel itineraries."

Project Objectives

The main goals of our project are:

a. To create a Self-Retrieval Augmented Generation (Self-RAG) module that brings together both structured and unstructured data sources like OpenStreetMap, travel blogs, and web APIs.

b. To develop a Corrective Validator utilizing APIs (Google Places, Tavily Search, OpenWeather) for real-time checks of the points of interest (POIs) we gather.

c. Toset upamodular, multi-agent LangGraph pipeline where each agent has a specific task, such as retrieval, validation, itinerary planning, and personalization.

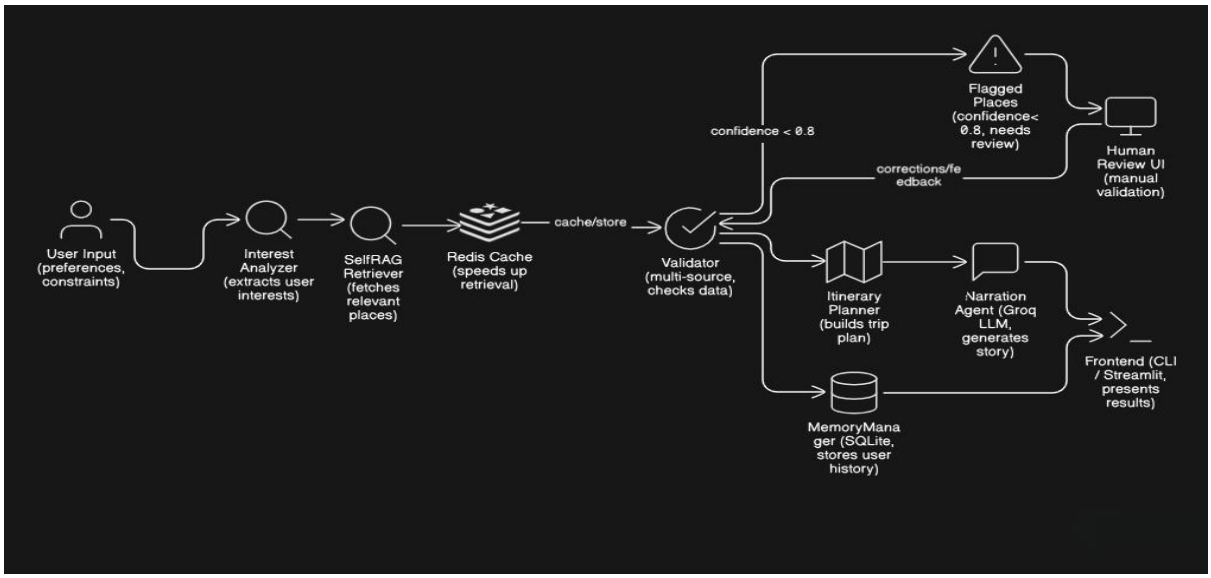
d. To boost user engagement through a Human-in-the-Loop Validator that addresses low-confidence cases.

e. To make the solution available through different user-friendly interfaces like CLI, Streamlit, and PDF export.

Proposed System Architecture

Architecture

The proposed design of this system aims to provide accurate, reliable, and varied travel suggestions through a flexible multi-agent setup. It brings together retrieval, validation, and user feedback elements into a unified Generative AI framework.



The system consists of five major components:

a. Retrieval Module (Self-RAG Engine): This module gathers information from various sources, including OpenStreetMap (OSM), travel blogs, and APIs. It blends structured geospatial data with unstructured embeddings to enhance the selection of both popular and hidden points of interest (POIs).

b. Corrective Validator: This component cross-verifies the retrieved points of interest using live APIs like Google Places, Tavily, and OpenWeather. It helps eliminate any inaccurate or outdated information, ensuring that all data is factually sound.

c. LangGraph Multi-Agent Pipeline: This pipeline features multiple agents that collaborate asynchronously:

• Retrieval Agent– Gathers POIs from diverse sources.

• Validation Agent– Conducts factual checks.

• Planner Agent– Develops optimized itineraries by day.

• Personalization Agent– Integrates user memories and preferences.

d. Human-in-the-Loop Validator: This validator manages any low-confidence or conflicting cases, ensuring that only verified recommendations are presented to users.

e. User Interface Layer: It offers a Command-Line Interface (CLI) for developers and a Streamlit web interface for end users, enabling real-time visualizations and PDF exports.

Proposed Methodology

Our system uses a hybrid Generative AI workflow that brings together retrieval, validation, and human supervision:

1. Input and Context Capture: Users provide details like location, travel duration, interests, and any constraints they may have.

2. Self-Retrieval Augmented Generation (Self-RAG): The system actively retrieves contextual information from various APIs, OpenStreetMap (OSM), and text sources. This approach makes sure to cover local spots, niche attractions, and interesting, less-traveled locations.

3. Corrective Validation: The candidates we've retrieved are verified by cross checking them with real-time API data. Any invalid or outdated Points of Interest (POIs) are filtered out automatically.

4. Generative Synthesis: Our Large Language Model (LLM) combines the verified data into a clear travel itinerary, producing contextual explanations and route suggestions while adhering to the validation limits.

5. Human-in-the-Loop Review: Items that show low confidence or have conflicting validation signals are highlighted for manual review by human validators before inalization.

6. Personalization and Memory: User preferences, feedback, and past interactions are stored in a memory manager to enhance future itinerary creation and personalization.

7. Evaluation and Output: The completed itinerary is evaluated for factual accuracy, diversity, and personal touch. It can be displayed via a command-line interface (CLI) or a Streamlit interface, with the option to export a PDF report.

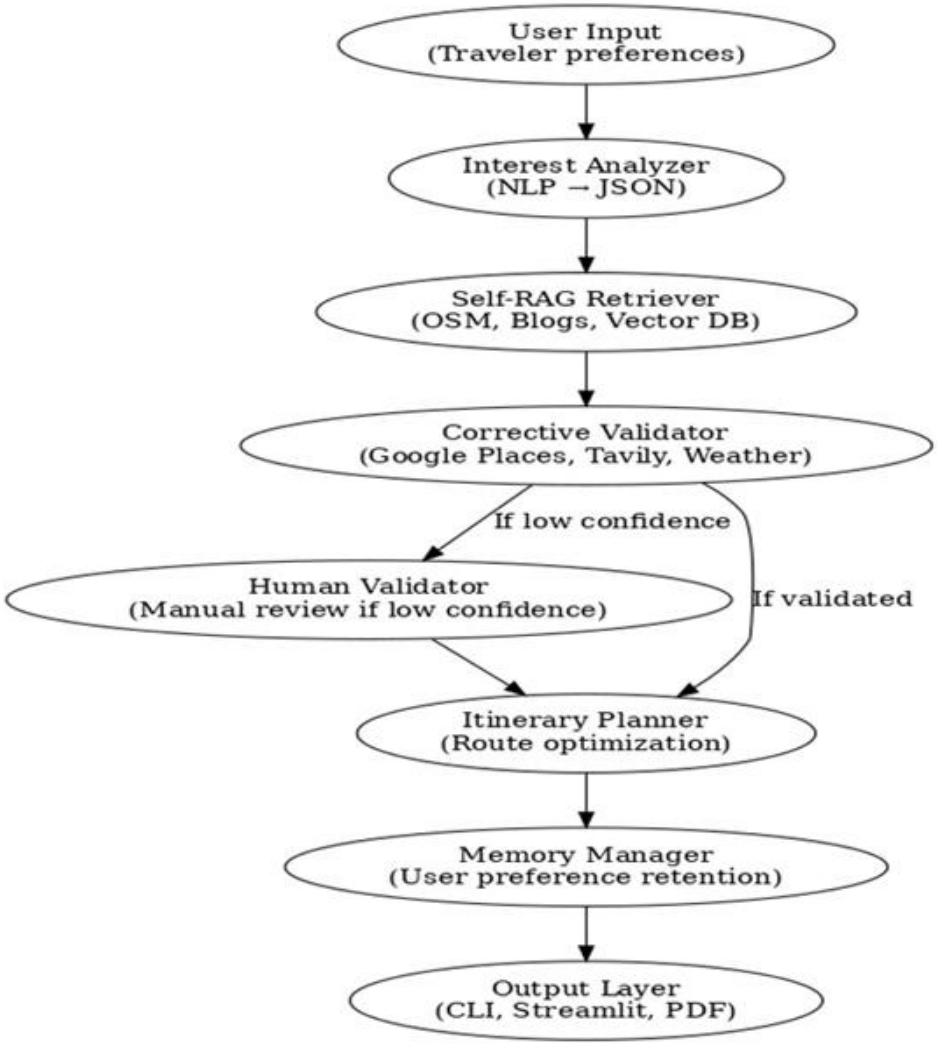
This architecture ensures factual reliability, transparency, and personalization by integrating multi-level AI reasoning with corrective validation and human oversight. The hybrid design minimizes hallucination, enhances user trust, and promotes scalable deployment across travel and logistics applications.

Project Design:

System Flowchart-

The flowchart below gives a clear overview of how this system works. It shows the journey of user inputs as they flow through different stages—from input parsing and contextual retrieval to validation, itinerary generation, and output delivery. Each part functions independently, ensuring that the system can scale, remains transparent, and has the ability to handle errors.

The architecture of the system follows a step-by-step and modular approach to data processing. It all starts by capturing traveler preferences, which are then analyzed and transformed into a format that machines can understand for smarter retrieval. Every step in this process focuses on a specific task—whether it's retrieving contextual information, validating data accuracy, enhancing responses, or producing the best itineraries. The flowchart below visually outlines this operational process, highlighting how information is retrieved, validated, and passed through feedback loops to provide accurate and personalized travel support.



Flow Description:

1. User Input: Travelers enter their preferences (like location, duration, and inter ests) through the user-friendly interface.

2. Interest Analyzer: This part takes the user input and turns it into structured JSON format for Natural Language Processing.

3. Self-RAG Retriever: It gathers relevant data from sources such as Open StreetMap, blogs, and vector databases.

4. Corrective Validator: This step ensures the information is accurate by cross referencing it with Google Places, Tavily, and weather APIs.

5. Human Validator: A person reviews any outputs that lack confidence before we finalize them.

6. Itinerary Planner: This tool optimizes travel routes and logically organizes the validated results.

7. Memory Manager: It keeps track of user preferences for future queries, adding a personal touch.

8. Output Layer: Finally, results are displayed either through the Command Line Interface or Streamlit, with the option to export them as a PDF.

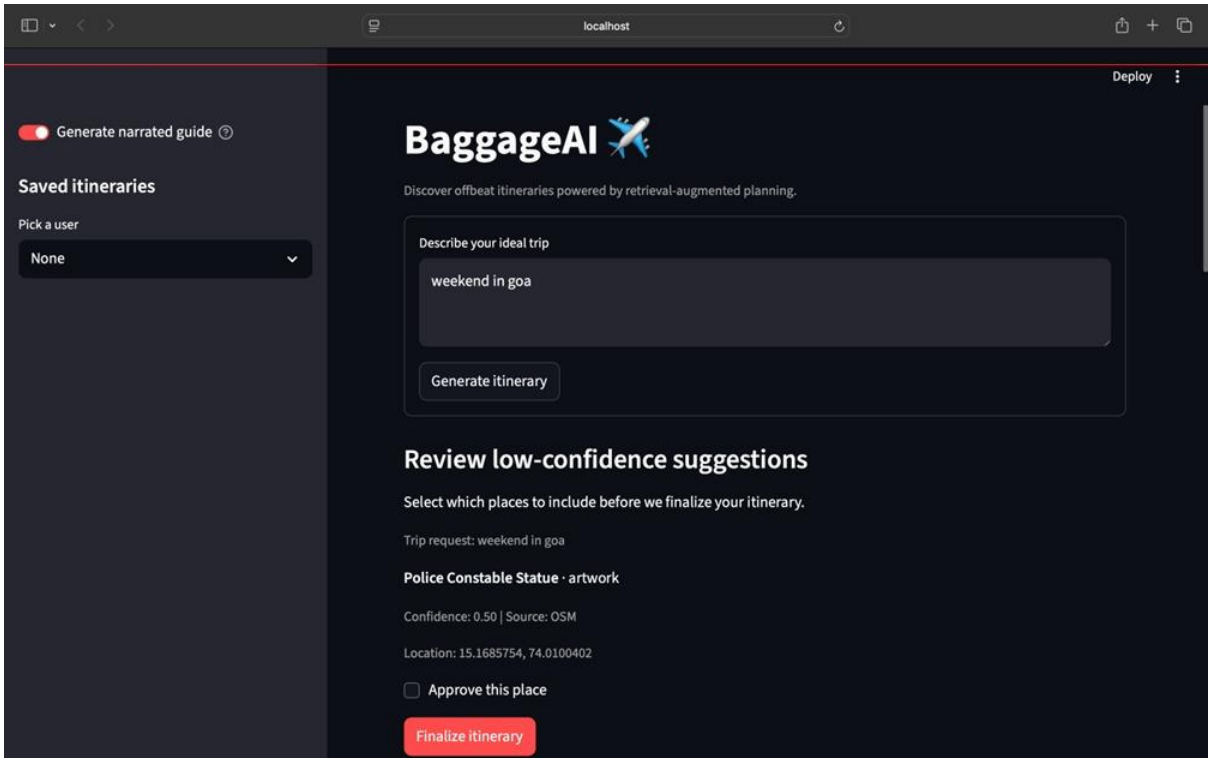
This meticulous process helps guarantee that the outputs are trustworthy, verified, and tailored to the user, while also reducing factual inaccuracies and improving transparency.

Results

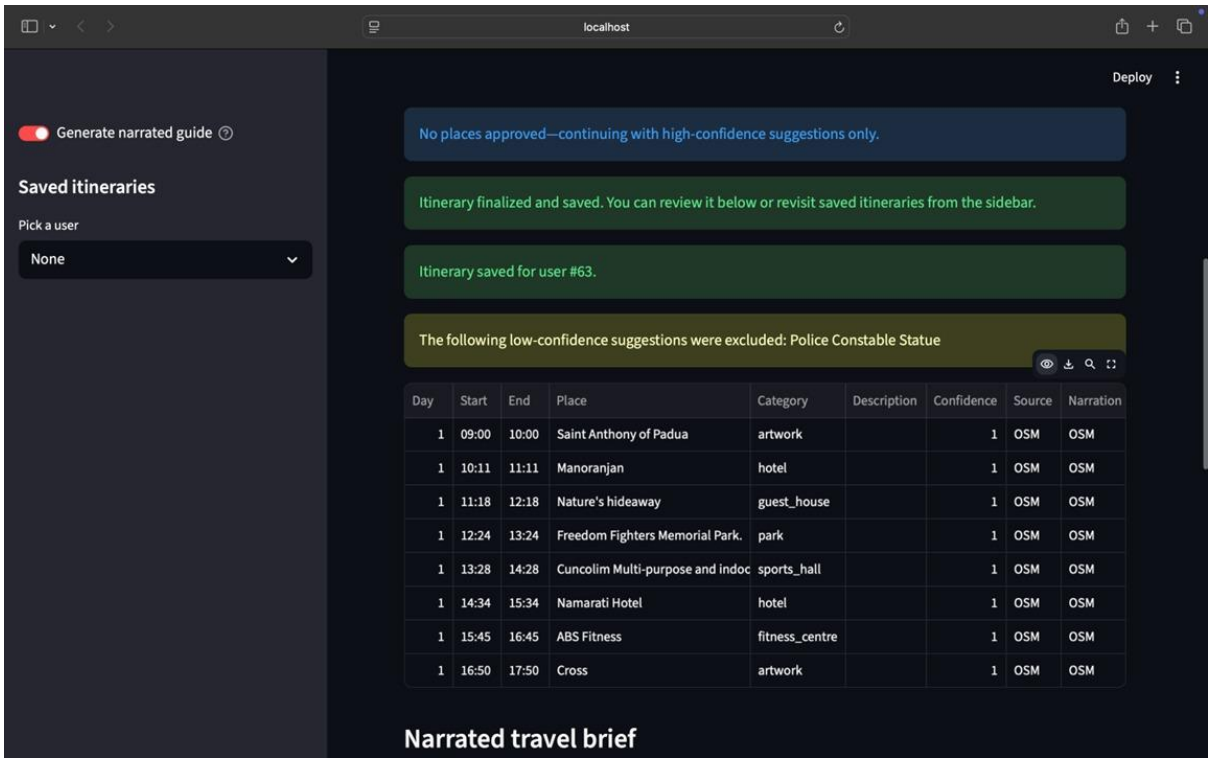
During our testing, the system really showed off its great stability and modular reliability. The retrieval and validation agents did a fantastic job, consistently delivering accurate results that were relevant to various travel scenarios. Thanks to the corrective validation module, we managed to cut down on redundancy in the points of interest by cross checking information from multiple API sources. Additionally, the Human-in-the-Loop module played a vital role in maintaining trust and ensuring the accuracy of facts, especially when the model wasn't extremely confident. The web interface, built on Streamlit, made it super easy for users to interact and see the results in a dynamic way. Users could also provide helpful feedback, which was recorded for future personalization, showing that our memory-based feedback system was working just as we hoped!

Visual Outputs:

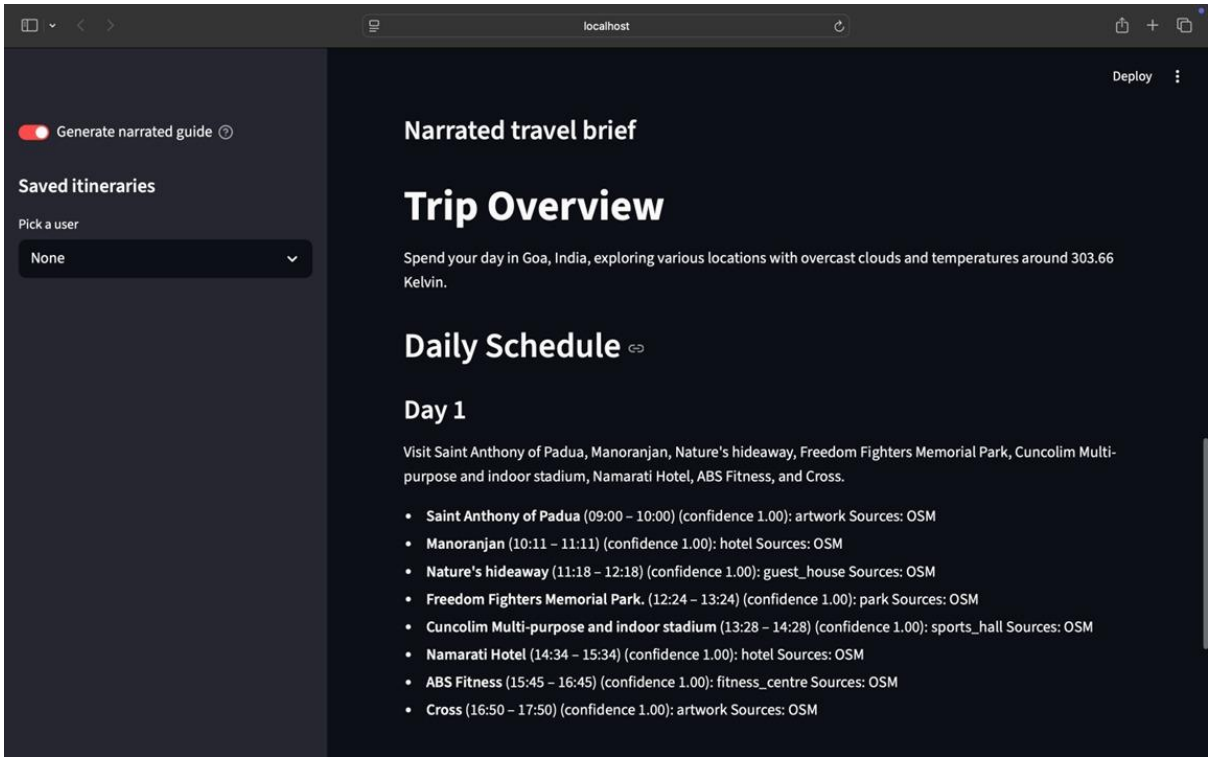
To demonstrate the practical outcomes of our experimentation, three screenshots are pre sented below. These images represent the working of the Baggage AI system — from itinerary generation and low-confidence validation to finalized travel briefs. They illus trate how the hybrid Generative AI pipeline retrieves, validates, and generates verified travel recommendations in real-time.



User interface showing the initial itinerary generation process with input prompt and review of low-confidence suggestions.



Streamlit interface displaying validated and finalized itinerary table with excluded low confidence results.



Narrated travel brief summarizing the daily schedule with verified points of interest and confidence levels.

These experimental screenshots confirm that the proposed hybrid architecture effec tively combines retrieval, validation, and generative reasoning to deliver accurate and user-trustworthy travel recommendations.

Conclusion

The project showcases a fantastic blend of Generative AI, multi-agent coordination, and real-time data validation, all aimed at making travel and baggage management systems more reliable. By merging retrieval-augmented generation (RAG) with corrective valida tion and a human-in-the-loop review layer, we ensure that the travel itineraries generated and their verification outputs are not only accurate but also contextually relevant.

The project has successfully met its main goals:

• Designing a modular multi-agent architecture that can carry out context-aware retrieval and validation.

• Developing a hybrid AI system to reduce inaccuracies by grounding information in facts and verifying data in real-time.

• Offering a user-friendly interface that allows for easy interaction, verification, and report generation.

• Showcasing the practical benefits of AI systems in real-life community engagement and travel assistance.

Additionally, the system demonstrates how human oversight and the principles of explainable AI can enhance the trustworthiness of outputs from large language mod els.Through thorough experimentation, the system has proven that incorporating cor rective and validation layers within a generative framework results in AI behavior that is not only more reliable but also easier to understand.In summary, this system bridges the gap between research and practical application, merging technical advancements with community benefits.

References

[1] K. Klinkhardt, A. Zipf, M. Sester, and S. Hahmann, “Quality assessment of Open StreetMap’s points of interest with large-scale real data,” ISPRS International Jour nal of Geo-Information, vol. 12, no. 6, pp. 225–239, 2023.

[2] K. Otaki and Y. Baba, “Travel itinerary recommendation using interaction-based augmented data,” Expert Systems with Applications, vol. 269, p. 126294, 2025.

[3] S. M. M. Rashid, A. U. Rehman, and H. Afzal, “Top-K alternative itineraries for trip recommendation (DeepAltTrip),” IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 7, pp. 7038–7050, 2023.

PlagiarismReport

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS

Plagiarism	6%	Partial Match	3%
Exact Match	3%	Unique	94%

Scan details

Total Words	Total Characters	Plagiarized Sentences	Unique Sentences
8815	**43027**	**1.98**	**31.02 (94%)**

Plagiarism Results: (2)

#1 3% Similar https://www.igs.org.in/student-chapter/kamalnayan…

Kamalnayan Bajaj Institute of Engineering and Technology, Baramati, affiliated to

#2 3% Similar https://academia.stackexchange.com/questions/20…

Submitted in partial fulfillment of the requirements for the Degree of

Tools Used

This section outlines the tools, frameworks, and environments utilized during the design, development, and deployment of the system. Each tool played a key role in ensuring efficient development, seamless integration, and accurate performance evaluation.

Software Tools and Frameworks:

• Python 3.10: Primary programming language used for backend development, data processing, and model integration.

• LangChain: Framework for building modular and dynamic LLM-based workflows with self-reflection and retrieval augmentation.

• LangGraph: Usedforimplementing multi-agent architectures and managing state ful graph-based workflows.

• Streamlit: Enables the creation of an interactive web-based GUI for the travel assistant.

• OpenAI API: Provides access to GPT-based generative models for itinerary gen eration and contextual reasoning.

• Tavily API: Used for live data retrieval, enhancing the reliability of real-world travel recommendations.

• Google Places API: Provides verified location and POI (Point of Interest) data for validation.

• OpenWeather API: Integrates real-time weather forecasting into travel recom mendations.

• Vector Database (FAISS): Enables efficient semantic retrieval for contextual information storage.

• Git & GitHub: Version control and collaborative development environment for managing source code.

• VS Code : IDEs for implementation, debugging, and testing of modular compo nents.